2025 23rd International Conference on ICT and Knowledge Engineering (ICT&KE)

Feasibility of Serial Split Inference with MobileNetV2 on ESP32-S3 for Real-Time Object Recognition

Prajin Palangsantikul
Graduate School of Information Technology
Siam University
Bangkok, Thailand
prajin.ast@gmail.com

Wipavan Narksarp
Graduate School of Information Technology
Siam University
Bangkok, Thailand
wipavan.nar@siam.edu

Wichian Premchaiswadi
Graduate School of Information Technology
Siam University
Bangkok, Thailand
wichian@siam.edu

2025 23rd International Conference on ICT and Knowledge Engineering (ICT&KE) | 979-8-3315-6540-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICTKE67052.2025.11274428

***Abstract*—This work investigates the feasibility of serial-based split inference for real-time object recognition applications on resource-constrained edge devices using dual ESP32-S3 units. MobileNetV2 with pretrained ImageNet weights is employed as a feature extractor, while transfer learning is applied to retrain a lightweight classifier for six target categories. To enable efficient deployment, the model is quantized into INT8 format. The proposed system uses an ESP32-S3-CAM to capture images and generate a [1,1280] INT8 feature vector, which is transmitted via UART to a second ESP32-S3 running a dedicated TFLite classifier. Experimental evaluation of model size, transmission latency, memory usage, and classification accuracy demonstrates the potential of this architecture for edge AI applications under strict resource constraints. While current implementation achieves 0.25 FPS, the study identifies optimization strategies required to achieve real-time performance (≥5 FPS) on microcontroller-based platforms.**

***Keywords*—Split Inference, MobileNetV2, Transfer Learning, Real-Time Object Recognition, TensorFlow Lite Micro, Edge AI, Quantization.**

I. Introduction

Deep learning has become the cornerstone of modern computer vision–based object recognition. However, deploying deep neural network models on embedded devices with strict constraints in power, computation, and memory remains a significant challenge [1]. To address this issue, research on lightweight convolutional neural network (CNN) architectures has emerged, aiming to enable efficient operation on resource-limited hardware. MobileNetV2, for example, introduces the concepts of *inverted residuals* and *linear bottlenecks* in conjunction with depthwise separable convolutions, effectively reducing computational cost while preserving accuracy [2], [3]. More recent work, such as MobileNetV3, leverages platform-aware neural architecture search (NAS) together with quantization-friendly activation functions (e.g., *hard-swish*) to further improve the accuracy–latency trade-off on real devices [5]. In addition, findings from the ShuffleNetV2 family highlight that floating-point operations (FLOPs) are not always reliable indicators of real-world speed, since performance is also influenced by factors such as memory access cost and parallelism [4].

Another crucial component for microcontroller deployment is integer quantization (INT8) and quantization-aware training (QAT), which systematically define scale and zero-point parameters. This enables inference to be performed entirely with integer arithmetic while maintaining accuracy close to that of floating-point computation [6]. In addition, model reduction strategies such as pruning and knowledge distillation have been widely employed to compress model size and transfer knowledge from larger networks to smaller ones under strict resource constraints [7], [8].

On the software side, microcontroller platforms are supported by frameworks such as TensorFlow Lite for Microcontrollers (TFLM), which employs an interpreter-based runtime and a statically allocated tensor arena (i.e., no dynamic memory allocation during execution). TFLM also incorporates a memory planner and supports integration of vendor-specific acceleration kernels, similar to the CMSIS-NN approach for Arm Cortex-M processors [14], [15]. From a system architecture perspective, the concept of split or collaborative inference distributes computational load between devices and the edge/cloud to balance latency and energy consumption. Pioneering works such as Neurosurgeon and subsequent efforts like Edgent have advanced this paradigm by combining partition point selection with early-exit mechanisms to adapt under variable network conditions [9]–[13].

Building upon this foundation, this study presents a feasibility analysis of serial-based split inference on dual ESP32-S3 devices. The first unit (ESP32-S3-CAM) extracts features using a quantized MobileNetV2-INT8 model—trained via transfer learning on ImageNet for six target classes—and transmits the resulting [1,1280] INT8 feature vector over UART to the second unit for real-time classification. The system is evaluated in terms of model size, transmission latency, memory utilization, and classification accuracy on actual hardware, thereby demonstrating the potential of edge AI deployment without reliance on IP-based networks.

979-8-3315-6540-4/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:41:35 UTC from IEEE Xplore. Restrictions apply.

II. Related Work

A. Lightweight CNNs for Embedded Devices

The MobileNetV2 family employs inverted residuals, linear bottlenecks, and depthwise separable convolutions to reduce parameters and computational cost while maintaining high accuracy [2], [3]. The ShuffleNetV2 approach proposes practical design guidelines—such as maintaining equal channel widths between input and output layers and avoiding excessive branching—to ensure *actual runtime efficiency* rather than merely achieving low FLOPs [4]. Meanwhile, MobileNetV3 leverages platform-aware neural architecture search (NAS) together with hard-swish activations and squeeze-and-excitation (SE) modules to optimize the accuracy–latency trade-off on real devices [5]. In this work, MobileNetV2 is selected as the feature extractor, given its compact bottleneck structure and intrinsic compatibility with INT8 quantization.

B. Model Compression and Integer-Only Inference

Integer-only inference (INT8) assigns scale and zero-point parameters to weights and activations (with biases stored in int32) and inserts fake quantization nodes during training. This approach enables an efficient trade-off between latency and accuracy on mobile and microcontroller platforms [6]. Iterative pruning can further reduce parameters and computational load by several folds while preserving accuracy [7]. Meanwhile, knowledge distillation transfers probabilistic knowledge within a teacher–student framework, allowing smaller models to retain performance comparable to larger ones [8]. In this work, however, emphasis is placed on INT8 post-training quantization (PTQ) and quantization-aware training (QAT) to align with the constraints and toolchains available on the ESP32-S3.

C. Split / Collaborative Inference

Neurosurgeon introduced layer-wise partitioning of neural networks between device and cloud by modeling per-layer latency and energy consumption to determine the *optimal split point* under latency and energy constraints [9]. Building upon this idea, Edgent proposed an on-demand co-inference framework that jointly optimizes partitioning and early-exit mechanisms under both static and dynamic conditions [10], [12]. The early-exit paradigm, exemplified by BranchyNet, integrates intermediate exit points based on model confidence to reduce average latency without significant loss of accuracy [11]. Another research direction involves feature/bottleneck compression, in which encoder–decoder modules are trained with channel-condition awareness to compress intermediate representations transmitted between devices and the edge (e.g., BottleNet++) [13]. In contrast to these approaches that rely on IP-based networking, the present work investigates UART-based MCU-to-MCU inference, which benefits from fixed latency and the absence of network stack overhead.

D. Feature Transmission and Bottleneck Design

A primary challenge in split inference lies in the size of intermediate feature vectors, motivating several studies to reduce or compress feature representations prior to transmission [11], [13]. A notable advantage of MobileNetV2 is its compact linear bottleneck, which enables the selection of a split point that produces a [1,1280] INT8 feature vector. This representation can be efficiently transmitted over UART and readily processed by the subsequent stage [2], [3].

E. Microcontroller Frameworks and Libraries

TensorFlow Lite for Microcontrollers (TFLM) employs an interpreter-based runtime along with a statically allocated tensor arena and a memory planner (including offline-planned layouts), enabling precise prediction of memory usage and latency with minimal overhead. Additionally, TFLM supports integration of vendor-specific acceleration kernels, similar to the CMSIS-NN approach for Arm processors [14], [15]. Moreover, Svoboda et al. [1] provide an in-depth analysis of the latency costs associated with data movement and quantization on microcontrollers, highlighting system-level constraints that developers must consider alongside framework selection. These features allow dual ESP32-S3 units to run the feature extractor and classifier separately with predictable and stable performance.

F. Positioning of This Work

In contrast to prior split/co-inference studies over Wi-Fi or mobile networks that focus on bandwidth-adaptive strategies [9], [10], [12], this work shifts attention to UART-based serial links, which provide fixed and predictable latency. Such an approach is particularly suitable for embedded and industrial applications that require stable and secure wired communication. The study reports practical metrics—including model size, memory usage, transmission latency, and post-INT8 accuracy—on actual hardware, serving as a reference for decisions regarding the deployment of serial-based split inference.

III. System Design

The proposed architecture is designed as a dual-board split inference pipeline on two ESP32-S3 boards, each equipped with a display. The system is divided into two main modules: (i) the Feature Extraction Module, running on an ESP32-S3-CAM with an integrated camera and a 2.4-inch TFT SPI 240×320 display, and (ii) the Classification Module, running on a separate ESP32-S3 board with its own display. Both modules are connected via a UART interface, as illustrated in Fig. 1. To support low-latency and predictable data transmission, the system resources used for developing this architecture are summarized in Table I. The table details the specifications of the ESP32-S3-CAM and ESP32-S3 boards, including input image size, processing model, and allocated memory, providing key reference information for system design and constraint definition.

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:41:35 UTC from IEEE Xplore. Restrictions apply.

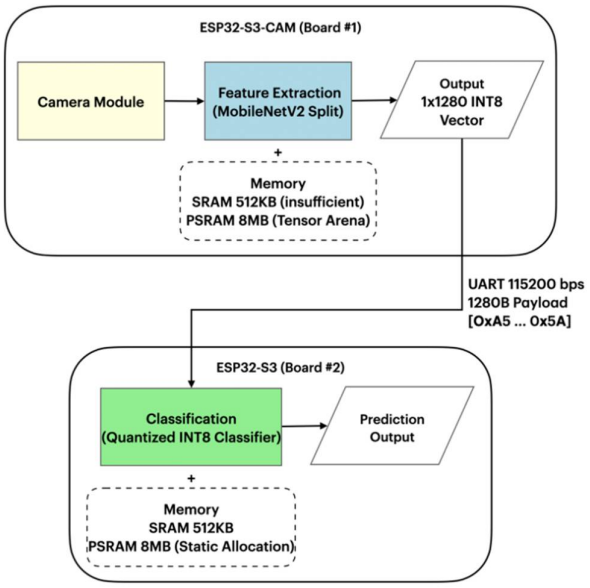


Fig. 1. System architecture of the dual-board ESP32-S3 implementation. The ESP32-S3-CAM (Board #1) captures images via the camera module and performs feature extraction using a split MobileNetV2 model. Due to the limited on-chip SRAM (512 KB), external PSRAM (8 MB) is utilized as the tensor arena. The extracted 1×1280 INT8 feature vector is transmitted via UART (115200 bps, 1280B payload) to the ESP32-S3 (Board #2), where a quantized INT8 classifier is executed with statically allocated memory (SRAM 512 KB and PSRAM 8 MB) to generate the final prediction output.

TABLE I. SYSTEM RESOURCES

Detail	Description
ESP32-S3 CAM	8 MB/512 KB SRAM + 8 MB PSRAM
ESP32-S3	8 MB/512 KB SRAM + 8 MB PSRAM
TFT LCD Display	2.4 TFT SPI 240x320 (ILI9341 / ST7789)
Image size (240x240 JPEG Format)	40 KB
Feature Vector size	1×1280 INT8 (1280 Bytes)
Features Model sizes (.tflite)	621 KB
Classifier Model sizes (.tflite)	339 KB
ESP32-S3 CAM Tensor arena (PSRAM)	1,877 KB
ESP32-S3 Tensor arena (PSRAM)	339 KB
UART transmission	115,200 bps

The key design principles are as follows:

1) Workload Partitioning: The MobileNetV2 model is split at the linear bottleneck layer to produce the smallest possible feature vector without compromising classification performance. This results in a 1×1280 INT8 descriptor, which is suitable for serial transmission under bandwidth constraints.

2) Memory-Aware Deployment: With only 512 KB of on-chip SRAM, the tensor arena for the model cannot be fully stored locally. Both the feature extractor and classifier are therefore allocated to 8 MB PSRAM, leveraging the static memory management mechanisms of TensorFlow Lite for Microcontrollers (TFLM) to avoid overhead from dynamic allocation.

3) Communication Protocol: A lightweight framing protocol is implemented, using a fixed payload size of 1280 bytes with a start marker (0xA5) and an end marker (0x5A) at a baud rate of 115,200 bps.

This architecture enhances error resilience and simplifies synchronization between the two boards. It enables the distribution of a quantized INT8 MobileNetV2 model across dual boards, where the camera-equipped board handles the computationally intensive feature extraction, and the second board is responsible for classification.

IV. IMPLEMENTATION

The model is developed using TensorFlow Lite for Microcontrollers (TFLM) as the inference framework on both the ESP32-S3-CAM and ESP32-S3 boards. The feature extraction module is based on the MobileNetV2 architecture, which underwent post-training INT8 quantization and was compiled into a .tflite file for deployment.

The training dataset consists of six object classes (Cleanser Bottle, Flash Drive, Mouse, Remote, Scissors, and Toothpaste Tube), together with a background class, as shown in Fig. 2. To improve robustness and generalization, extensive data augmentation techniques are employed, including rotation, scaling/zooming, brightness/contrast/color adjustment, the addition of noise, random background replacement (solid color or blur), and minor shear and perspective transformations.

The training process follows a transfer learning approach, and the resulting model can be deployed within the Arduino IDE environment. Subsequently, the quantized and compiled model was implemented on the hardware platform. The ESP32-S3-CAM board performs feature extraction and transmits the resulting feature vector via UART to the ESP32-S3 board, which executes classification and displays the prediction output on a TFT screen, as illustrated in Fig. 3, showing the actual hardware implementation of the system.



Fig. 2. Images of the 6 classes (Cleanser Bottle, Flash Drive, Mouse, Remote, Scissors, and Toothpaste) along with the background images used for transfer learning.

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:41:35 UTC from IEEE Xplore. Restrictions apply.

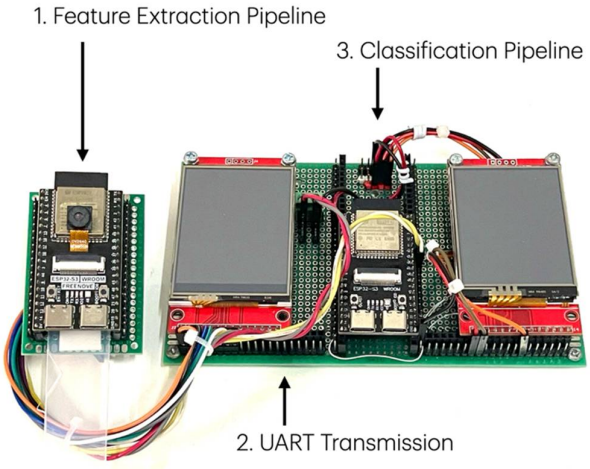


Fig. 3. shows the hardware setup of the ESP32-S3 CAM and ESP32-S3 boards used in the split inference pipeline implementation.

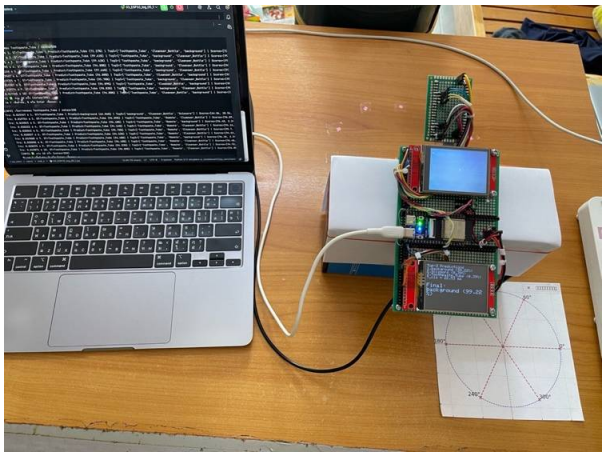


Fig. 4. Experimental setup. After the classification is performed, the prediction results are transmitted via the serial port to a computer, where they are recorded in .log files according to the predefined class labels.

The system operates in three main stages:

1) Feature Extraction Pipeline

a) Images are captured at a resolution of 224×224×3 using the ESP32-S3-CAM.

b) Pixel values are normalized by subtracting 128 to match the expected input for INT8 quantization (scale=1.0, zero_point=−128).

c) The quantized feature extractor generates a [1,1280] INT8 feature vector.

2) UART Transmission

a) The feature vector is serialized into a fixed-size payload of 1280 bytes.

b) UART transmission employs a framing protocol with start and end markers to ensure synchronization and prevent errors due to data misalignment.

3) Classification Pipeline

a) The second ESP32-S3 board receives the payload and deserializes it into the tensor arena allocated in PSRAM.

b) The classifier, trained on six object classes (Cleanser Bottle, Flash Drive, Mouse, Remote, Scissors, and Toothpaste Tube), performs inference on the feature vector.

c) Classification results are output as class probabilities, and predicted labels are sent to the host for evaluation.

The system was tested on 120 frames to measure latency, memory usage, and classification accuracy. The modular design allows for flexible adjustment of the classifier or split point without altering the communication mechanism.

V. Experimental Setup

To evaluate the proposed architecture, a split inference pipeline was implemented on two ESP32-S3 boards, dividing the workload into a feature extractor and a classifier, as illustrated in Fig. 4, consistent with the design described in Section III: System Design.

1) Hardware Configuration

a) The Feature Extractor runs on an ESP32-S3-CAM with an integrated camera, capturing images directly, which are also displayed on its onboard screen.

b) The Classifier runs on a separate ESP32-S3 board, connected to the feature extractor via a UART interface, and displays predicted results on its display.

c) Due to the 512 KB limit of on-chip SRAM, the tensor arena for both modules is allocated on 8 MB PSRAM.

2) Input Data Preparation

a) Captured images are resized to 224×224×3 to match the model input.

b) Since the INT8 model uses scale = 1.0 and zero_point = −128, 8-bit pixel values are normalized by subtracting 128 prior to inference.

3) Data Communication

a) Feature vectors from the MobileNetV2 linear bottleneck layer have a size of [1,1280] INT8 and are serialized into a fixed-size payload of 1280 bytes.

b) UART transmission uses a baud rate of 115,200 bps with a framing protocol including a start byte (0xA5) and end byte (0x5A) to reduce framing errors and ensure stable synchronization.

4) Testing Procedure

a) The system was evaluated over 120 consecutive frames captured at angles ranging from 0 to 360 degrees, with 30-degree increments.

b) Recorded variables include feature extraction latency, UART transmission time, end-to-end latency, memory usage, and classification accuracy.

c) All measurements were performed on actual hardware to reflect real-world performance under embedded system constraints (Fig. 5).

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:41:35 UTC from IEEE Xplore. Restrictions apply.

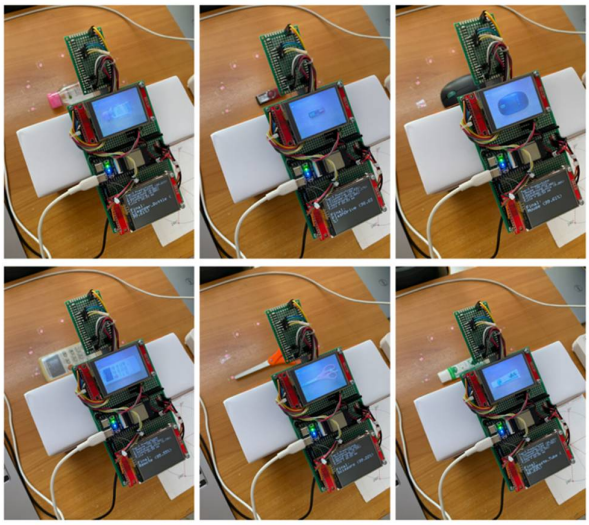


Fig. 5. Experimental setup at a 180-degree orientation, illustrating the placement of equipment for all six classes.

The system was evaluated by processing 120 consecutive frames to ensure statistical reliability. Using multiple frames allows for more accurate measurement of feature extraction latency, UART transmission time, and end-to-end latency, as the mean and variance better reflect system performance compared to single-frame tests. This approach also enables assessment of the system's stability in continuous object recognition, as well as evaluation of model performance metrics such as accuracy and F1-score under real-world operating conditions.

TABLE II. LATENCY BREAKDOWN (MS)

T_{cap}	T_{prep}	T_{ext}	T_{uart}	T_{cls}	T_{e2e}	FPS
112.26	12.53	3747.72	102.33	22.41	3997.37	0.25

Table II summarizes the measured latency of the proposed split inference system, including image capture time (T_{cap}), preprocessing time (T_{prep}), feature extraction time (T_{ext}), UART transmission time (T_{uart}), and classification time (T_{cls}). The end-to-end latency (T_{e2e}) is obtained by summing all components, yielding an average of approximately 3,997.37 ms per frame, corresponding to a processing speed of 0.25 FPS. The results indicate that feature extraction is the primary contributor to system delay, highlighting that performance improvements in this stage are critical for enabling real-time operation.

TABLE III. ACCURACY (6 CLASSES)

Class	Precision (%)	Recall	F1	Support
Cleanser_Bottle	0.8633	1.00	0.9266	120
FlashDrive	1.00	0.9917	0.9958	120
Mouse	1.00	0.9667	0.9831	120
Remote	1.00	0.9	0.9474	120
Scissors	1.00	0.9833	0.9916	120
Toothpaste_Tube	1.00	0.9250	0.9610	120
Accuracy	0.9611	0.9611	0.9611	0.9611
macro avg	0.8376	0.8238	0.8294	720
weighted avg	0.9772	0.9611	0.9676	720

Table III presents the object classification results of the proposed model across six classes, evaluated using Precision, Recall, and F1-score. Each class includes 120 test samples, as indicated in the Support column. Experimental results show that most classes achieve a Precision of 1.00, while Recall ranges from 0.90 to 1.00. The overall system Accuracy is 96.11%, with a macro-averaged F1-score of 0.8294 and a weighted F1-score of 0.9676. These results demonstrate that the proposed split inference system can maintain high classification performance even when deployed on hardware with strict resource constraints.

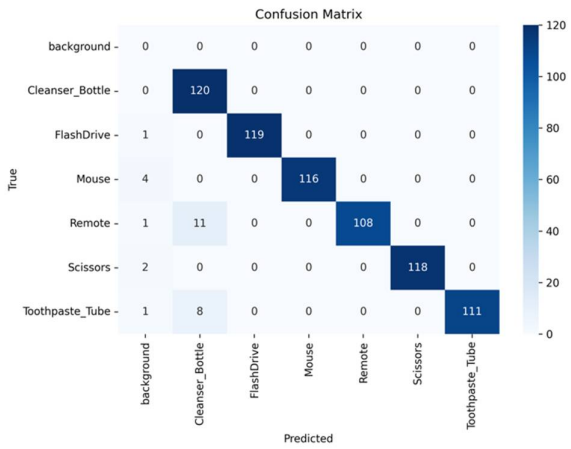


Fig. 6. Confusion matrix

Fig. 6 shows the confusion matrix for six-class object classification using the proposed split inference system. Rows correspond to actual labels, while columns represent predicted labels. Each class was evaluated with 120 test samples. Diagonal entries indicate correctly classified samples: Cleanser_Bottle – 120, FlashDrive – 119, Mouse – 116, Remote – 108, Scissors – 118, and Toothpaste_Tube – 111. Off-diagonal entries correspond to misclassifications, such as Remote misclassified as FlashDrive (11 samples) and Mouse misclassified as Cleanser_Bottle (4 samples). Overall, the results confirm that the model performs effectively, with most errors occurring between visually similar classes.

VI. RESULTS AND DISCUSSION

The experimental results demonstrate the feasibility of serial-based split inference on dual ESP32-S3 boards for near real-time object classification under embedded system constraints. Although the system relies on PSRAM for memory allocation and utilizes reference TFLM kernels without vendor-specific optimization, it remains stable and reliable in operation.

A. Latency and Throughput

Table II presents the measured latency of the proposed dual-board split inference system, including image capture time (T_{cap}), preprocessing time (T_{prep}), feature extraction time (T_{ext}), UART transmission time (T_{uart}), and classification time (T_{cls}). The end-to-end latency (T_{e2e}) averages 3,997.37 ms per frame, corresponding to a processing speed of 0.25 FPS. The results indicate that feature extraction is the primary contributor to system delay, highlighting that improvements in this stage are critical for achieving real-time performance.

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:41:35 UTC from IEEE Xplore. Restrictions apply.

B. Accuracy

Table III summarizes the classification performance of the system across six classes, evaluated using Precision, Recall, and F1-score with 120 test samples per class. The system achieves an overall Accuracy of 96.11%, with a macro-averaged F1-score of 0.8294 and a weighted F1-score of 0.9676. These results confirm that the proposed system can maintain high classification performance even when deployed on hardware with strict resource constraints.

C. Confusion Matrix Analysis

Fig. 6 shows the confusion matrix for six-class object classification using the proposed split inference system. Diagonal entries represent correctly classified samples: Cleanser_Bottle – 120, FlashDrive – 119, Mouse – 116, Remote – 108, Scissors – 118, and Toothpaste_Tube – 111. Most misclassifications occur between visually similar classes, such as Remote misclassified as FlashDrive (11 samples) and Mouse misclassified as Cleanser_Bottle (4 samples). Overall, the confusion matrix indicates that the system achieves high classification performance, with errors primarily arising from classes with close visual resemblance.

D. Bottlenecks

The primary bottlenecks of the system arise from two key factors: (i) the PSRAM bandwidth, which limits data access speed compared to on-chip SRAM, and (ii) the computational complexity of depthwise and pointwise convolutions in the feature extraction module. Nevertheless, selecting the linear bottleneck layer of MobileNetV2 as the split point produces a compact 1×1280 INT8 feature vector, reducing communication overhead and balancing computation and transmission time.

E. Future Improvement Strategies

Although this work primarily focuses on feasibility, several strategies could further improve system performance in future implementations, including:

- Reducing input resolution to lower the computational load during feature extraction.
- Adjusting the width multiplier (α) to reduce the number of channels and the size of the feature vector.
- Using vendor-optimized kernels to accelerate convolution operations and decrease overall latency.

F. Summary of Discussion

In summary, the experimental results confirm that UART-based split inference is suitable for embedded real-time object recognition requiring stable and predictable latency. Careful selection of the split point and efficient payload management are key factors enabling practical operation of the system under hardware constraints.

VII. Conclusion and Future Work

This study presented a feasibility study of split inference on dual ESP32-S3 boards, using a UART interface for real-time object classification under embedded system constraints. The MobileNetV2 model, quantized to INT8, was divided into two parts: a feature extractor running on the ESP32-S3-CAM and a classifier running on a separate ESP32-S3 board. Experimental results confirm that this architecture operates stably, with end-to-end latency suitable for near real-time operation, while maintaining satisfactory classification accuracy.

Key findings include:

1) Selecting the linear bottleneck layer of MobileNetV2 as the split point produces a compact feature vector, suitable for bandwidth-limited serial communication.

2) PSRAM memory allocation, despite its latency limitations, can support the processing of a medium-sized model.

3) The lightweight UART framing protocol ensures reliable and predictable data transmission.

Future research directions include:

- Reducing input resolution or width multiplier (α) to lower computational load and overall latency.
- Leveraging vendor-optimized kernels to accelerate convolutions and reduce processing time.
- Extending to multi-node split inference with more than two boards to handle more complex tasks, investigating synchronization and scheduling mechanisms for distributed systems.
- Evaluating the system on larger and more diverse datasets to assess model generalization capability.

In conclusion, the results demonstrate that split inference on dual microcontrollers via UART is not only feasible but also holds potential for embedded real-time AI applications requiring stability, security, and reliable operation under strict resource constraints.

References

[1] F. Svoboda, J. Fernandez-Marques, E. Liberis, and N. D. Lane, "Deep Learning on Microcontrollers: A Study on Deployment Costs and Challenges," in *Proc. 2nd European Workshop on Machine Learning and Systems (EuroMLSys'22)*, Rennes, France, Apr. 2022, pp. 1–10. doi: 10.1145/3517207.3526978.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Inverted Residuals and Linear Bottlenecks: Mobile Networks for Classification, Detection and Segmentation (MobileNetV2)," 2018. (Tech. report/early version corresponding to MobileNetV2.)

[3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *arXiv:1801.04381*, v4, 2019.

[4] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," 2018. (ECCV 2018 / arXiv:1807.11164.)

[5] A. Howard, M. Sandler, G. Chu, L.-C. Chen, et al., "Searching for MobileNetV3," in *Proc. ICCV*, 2019.

[6] B. Jacob, S. Kligys, B. Chen, M. Zhu, et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. CVPR*, 2018.

[7] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both Weights and Connections for Efficient Neural Networks," 2015. (arXiv:1506.02626.)

[8] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," 2015. (arXiv:1503.02531.)

[9] (UMich Clarity Lab) Y. Kang, J. Hauswald, C. Gao, et al., "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge," in *Proc. ASPLOS*, 2017.

[10] E. Li, Z. Zhou, and X. Chen, "Edge Intelligence: On-Demand Deep Learning Model Co-Inference with Device-Edge Synergy (Edgent),"

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:41:35 UTC from IEEE Xplore. Restrictions apply.

in *Proc. SIGCOMM Workshop on Mobile Edge Communications (MEC)*, 2018.

[11] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks," 2017. (arXiv:1709.01686.)

[12] E. Li, C. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing (Edgent)," *IEEE Trans. Wireless Commun.*, preprint, 2019.

[13] J. Shao and J. Zhang, "BottleNet++: An End-to-End Approach for Feature Compression in Device-Edge Co-Inference Systems," *arXiv:1910.14315*, v5, 2020.

[14] R. David, J. Duke, A. Jain, V. J. Janapa Reddi, N. Jeffries, J. Li, … P. Warden, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," in *Proc. MLSys*, 2021.

[15] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs," 2018. (arXiv:1801.06601.)

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:41:35 UTC from IEEE Xplore. Restrictions apply.